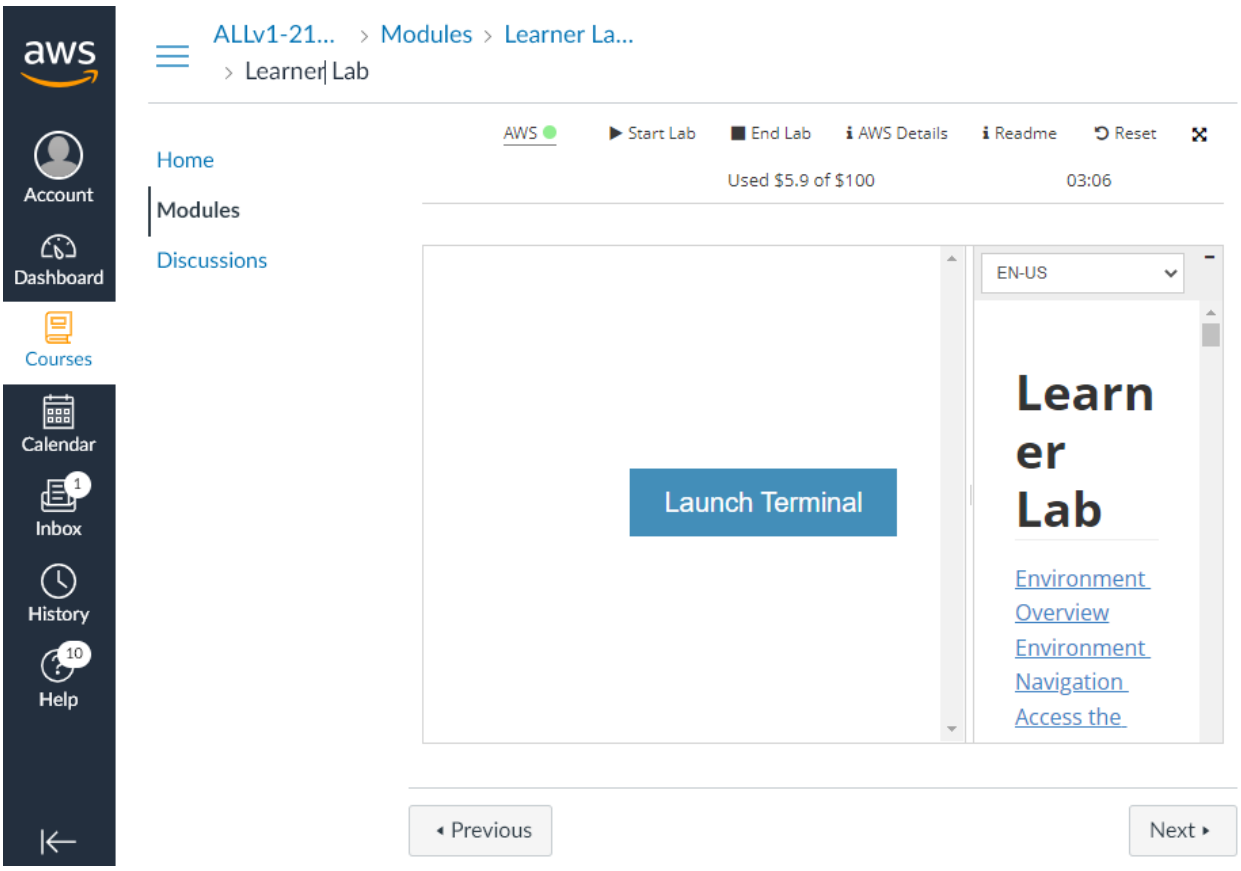
Cloud Essentails

1. AWS introduction

Aws Academy

For the introduction lab you will setup your Learner Lab environment. Navigate to the Learner lab module in the AWS academy Learner lab course. Press the play button to start the lab environment, a 4 hour timer will start:



You can extend the 4 hour timer by clicking the play button again. After the timer ends, all active services will shutdown but progress is saved.

Using AWS

AWS management console

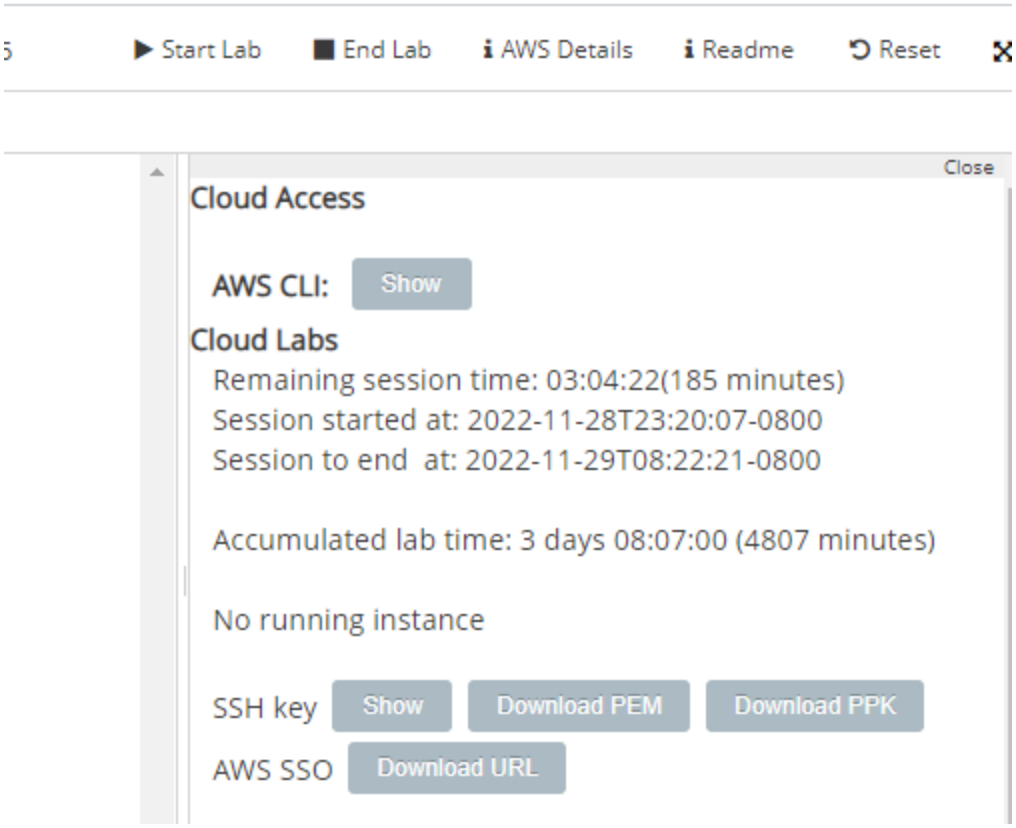
You can acces the AWS console application by pressing the

```
AWS
```

button. You can also navigate to the

```
AWS Details
```

tab where you can find the secret keys and ssh keys that we will use later on for accessing machines and the CLI:



You will notice that some services are restricted in the learner lab environment. As an alternative you can always use the **pluralsight sandbox** environment. The downside is that the sandbox resets after every use.

Using the AWS CLI

The AWS management console website might be a bit overwhelming when you first learn using cloud services. Using the CLI is an alternative way to create, change and manage AWS services. There are currently 2 versions of the CLI. For this course we will focus on using *AWS CLI version 2*.

Installing the CLI

Install the CLI using the urls below:

- **Windows 64 bit**
- **Mac OS PKG**
- **Linux installer**

To confirm the installation, open a terminal window and run the `aws --version` command.

Configuring the CLI

For genreal use, the `aws` CLI needs some information to actually be able to run:

- Access key ID
- Secret access key
- Session token
- AWS Region
- Output format

You can find the Access key ID and secret access key on the AWS academy learner lab page under the link *AWS Details*.

The CLI stores this information in a *profile* named `default` in a `credentials` file. We can manually create this file (usually located in the file `~/.aws/credentials`) or we can use the `aws configure` CLI command:

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYE
XAMPLEKEY
Default region name [None]: us-east-1
```

```
Default output format [None]: jsonCopy to clipboardErrorCopie
d
```

> Note that you will have to manually add the aws_session_token to the ~/.aws/credentials file when using AWS academy!

After this configuration you are ready to use the CLI tool to run commands. We will use the CLI in the upcoming chapters aswell as the AWS management console.

Testing the CLI

You can quickly test if your `~/.aws/credentials` configuration works with AWS Academy by using this command:

```
aws sts get-caller-identityCopy to clipboardErrorCopied
```

Exercises

1. Start your AWS learner lab and configure your environment so you can use the AWS CLI. After configuring the AWS credentials run the following command:

   ```
   aws ec2 describe-instance-types --query 'InstanceTypes[?st
   arts_with(InstanceType, `t`)].[InstanceType, ProcessorInf
   o, MemoryInfo]'Copy to clipboardErrorCopied
   ```

2. In the folder `~/.aws/` create a new file called `config` and add the following contents:

   ```
   [default]
   output = json
   region = us-east-1Copy to clipboardErrorCopied
   ```

   Run the command from exercise 1 again. Did the output change in any way? What other output options are there?

2. Compute

Exercise 1

Configure your environment so you can use the AWS CLI. When the CLI is configured run the following commands:

```
$ wget https://2tin-cloudessentials-resources.github.io/cloud
-essentials-labs/debuglabs/2_1.yml -outFile 2_1.yml
$ aws cloudformation deploy --template-file ./2_1.yml --stack
-name ch2ex1Copy to clipboardErrorCopied
```

Note: These commands work in Powershell, for Linux you might have to change wget options.

Using the AWS management console or CLI, try answering the following questions:

- What are the names of the 2 instances created?
- What is the name of the AMI that is used for both instances?
- What is the instance type for both machines?
- What is the public IPv4 address for `InstA`?
- What is the private IPv4 address for `InstB`?
- What is the name for the keypair linked to both machines?
- Search for a linked security group. What is the name of this group?
- What rules are present in this security group?
- Our customer is complaining that the costs for his EC2 instances are expensive. What causes this issue and how would you fix this?

When you are done you can run the command below to destroy resources used by this exercise:

```
aws cloudformation delete-stack --stack-name ch2ex1Copy to cl
```

```
ipboardErrorCopied
```

Exercise 2

Configure your environment so you can use the AWS CLI. When the CLI is configured run the following command:

```
$ wget https://2tin-cloudessentials-resources.github.io/cloud
-essentials-labs/debuglabs/2_2.yml -outFile 2_2.yml
$ aws cloudformation deploy --template-file ./2_2.yml --stack
-name ch2ex2Copy to clipboardErrorCopied
```

A) The commands above set up an EC2 instance named "imbroken". The instance uses a UserData script to install a default webserver but something went wrong. Using the `cloud-init status` command and `cloud-init-output.log` file try to figure out what went wrong. You can use the `vockey` to connect to the instance.

B) Using **This documentation page**, try to fix the issue and start the webserver. You should be able to see a web page when you visit the public IP address of the instance.

When you are done you can run the command below to destroy resources used by this exercise:

```
aws cloudformation delete-stack --stack-name ch2ex2Copy to cl
ipboardErrorCopied
```

Exercise 3

Create a new `t2.micro` instance using the Amazon linux AMI. Use a userdata script that automaticly installs `docker` on the machine.

Connect to the machine and run the `docker ps` command to verify the installation is succesful. Try running the **Docker hello world** example.

Exercise 4

Using the machine from *exercise 3*, try to run a Minecraft server using **Docker**.

Make sure you create / update the security group so that people can join your Minecraft server over the internet.

3. Storage

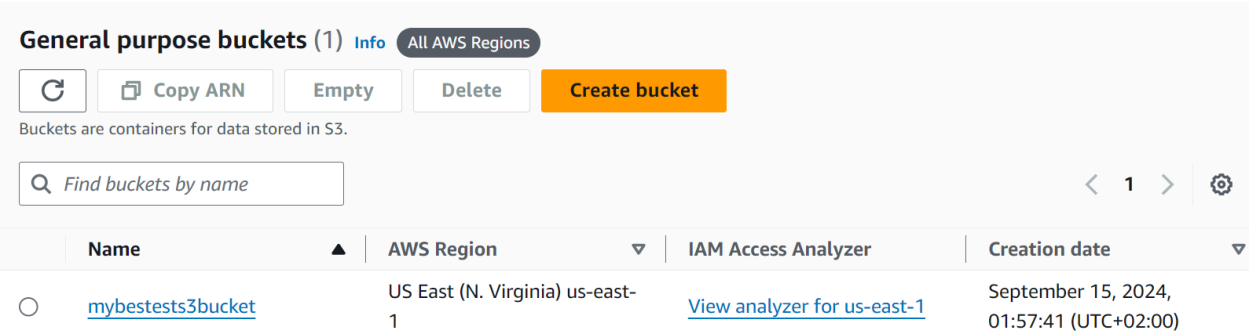
Exercise 1: Creating a Default S3 Bucket for Internal Data Storage

In this exercise, we will create a default S3 bucket to store internal data, such as logs and source code for a Lambda function (we'll discuss Lambda in more detail later). Follow these steps:

1. **Open the AWS Console** and navigate to the **S3 service**.
2. Click **Create Bucket**.
3. Give your bucket a **unique name**.
4. Leave the region set to **us-east-1** (the default region).

   Now, configure the access permissions:
5. Keep **"ACLs disabled (recommended)"** selected.
6. Ensure **"Block all public access"** is enabled, as this bucket is for internal use and does not require public access. *Note: This setting offers fine-grained control over public actions if you choose to change it later.*
7. Leave the default settings for **versioning, tagging, and encryption**.
8. Finally, click **Create Bucket** at the bottom of the screen.

And that's it—your bucket is now created!

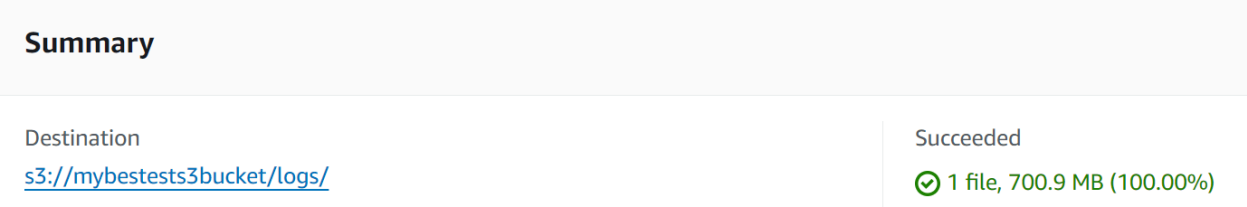


Uploading Files

Once your bucket is created, you can upload files to it. Each file will automatically be assigned **S3 and HTTP endpoints**, which you can use.

Try this:

1. In the bucket overview, create a folder named **"logs"**.
2. Upload a file (e.g., a zip file) from your laptop into the **logs** folder via the AWS Console.
3. After uploading, notice that the file appears in the bucket, and you can view its S3 and HTTP endpoints.



To verify your setup, log into the **CloudShell** service in the console and run the following commands to list your bucket:

```
aws s3 ls
aws s3 ls s3://my-bucket/my-folder/Copy to clipboardErrorCopi
ed
```

For example:

```
$ aws s3 ls
2024-09-15 01:57:41 mybestests3bucket
$ aws s3 ls s3://mybestests3bucket/logs/
2024-09-15 02:01:04          0
2024-09-15 02:03:19  734990917 cudnn.zipCopy to clipboardErro
rCopied
```

Exercise 2: Setting Up a Static Website on S3 Using the AWS CLI

In this exercise, you'll use the AWS CLI to create a new S3 bucket and configure it to host a static website. Follow the steps below:

1. **Create a new S3 bucket** Use the AWS CLI to create a new bucket with a name of your choice. This will host your static website.
2. **Enable Static Website Hosting** Enable static site hosting using the AWS CLI with the `aws s3 website` command. *Tip: You can configure error and redirect rules, but we will skip those for this lab.*
3. **Disable Block Public Access** Since public access is blocked by default, use the CLI to disable this setting so the static site can be publicly accessed. *Tip: Use the `aws s3api put-public-access-block` command to modify the access settings.*
4. **Add a Public Read Policy** Attach the following public read policy to allow access to the objects in the bucket:

   ```
   {
     "Version": "2012-10-17",
     "Statement": [
       {
         "Sid": "PublicReadGetObject",
         "Effect": "Allow",
         "Principal": "*",
         "Action": "s3:GetObject",
         "Resource": "arn:aws:s3:::bucket-name/*"
       }
     ]
   }Copy to clipboardErrorCopied
   ```

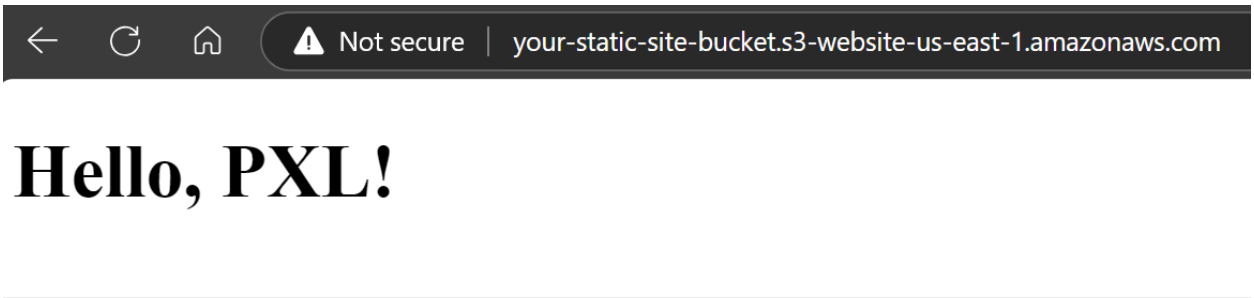
   Make sure to replace `bucket-name` with the actual name of your bucket.
5. **Upload Your Website Content** Create a simple `index.html` file (e.g., "Hello, world!") and sync it to your bucket using the AWS CLI.

6. **Test Your Static Website** After the file is uploaded, visit the website URL provided by the S3 bucket to view your static website. You should see your `index.html` content displayed.

End Result

By the end of this exercise, you should have:

1. A bucket containing a file and a **"logs"** directory.
2. A second bucket hosting a static website displaying your **index.html** file.



Advanced Exercise: Enabling Bucket Versioning and Using Pre-Signed URLs

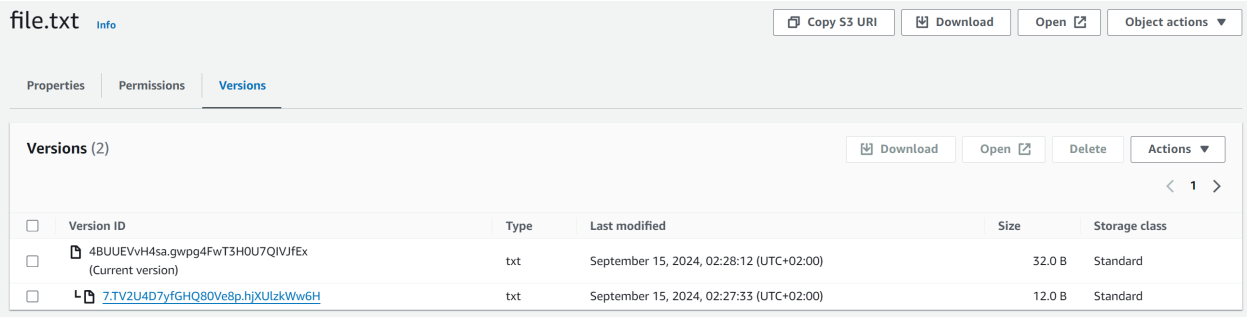
Step 1: Enable Bucket Versioning

1. Create a new S3 bucket (or use an existing one).
   - Make sure your bucket name is globally unique and doesn't contain spaces.
2. Enable versioning on your bucket using the AWS CLI.
   - Look for a command that starts with `put-bucket-versioning` to enable this feature.
3. Upload a file to your bucket.
   - Use the `aws s3 cp` command to upload files.
4. Modify the file on your local machine (e.g., add or remove text) and upload it again.

- Ensure the file has the same name when re-uploading so it overwrites the existing one.

5. List all the versions of the file stored in your bucket.
   - You can find an AWS CLI command that lists object versions by searching for `list-object-versions`.

After re-uploading the modified file, there should be multiple versions of it in the bucket.



Step 2: Generate Pre-Signed URLs

1. Upload a private file to your bucket.
   - The bucket should still have public access blocked, so the file is not available to the public.
2. Generate a pre-signed URL for the file that is valid for 60 minutes.
   - Search for a command that begins with `presign`. You can set the expiration time in seconds (e.g., 3600 for 1 hour).
3. Test the URL by opening it in a browser or sharing it with someone.
   - Once you generate the URL, paste it into your browser to see if you can access the file.
   - For example:

```
$ aws s3 presign ...
https://your-versioned-bucket.s3.us-east-1.amazonaws.com/g
itkraken-license.zip?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Am
z-Credential=ASIAUDXB3VYCREJPOJ7C%2F20240915%2Fus-east-1%2
```

```
Fs3%2Faws4_request&X-Amz-Date=20240915T003809Z&X-Amz-Expir
es=3600&X-Amz-SignedHeaders=host&X-Amz-Security-Token=IQoJ
b3JpZ2luX2VjELj%2F%2F%2F%2F%2F%2F%2F%2F%2F%2FwEaCXVzLXdlc3
QtMiJHMEUCIQDZH%2Bi5kXIh%2BtP7zLZQtcFlZoN%2BjILr3BlnuiDtoo
tloQIgBqf9qezvsvFrMYEV9%2Ff%2FTLnE4Jl6HD7tgbvmEilrQ84qugII
4f%2F%2F%2F%2F%2F%2F%2F%2F%2F%2FARABGgwyODI4Njc3NzI5MzMiDE
VIaxzXH1YER1JX%2BCqOApgSj8oV45seAQNSy8PIDB2BmhPm0mccEKH8tt
tA9HvY8dlwjKbU3kbYBgCrTYQPhZ7BG5r1z3NUukCpGa2MAPz7S8mXEH3Y
KWJm4GUJsUK%2FAZKsZ9niPzj6bbJnI2WekTDa0aBpcbrmeGHoR9mVM7aW
50DB9TTfuJpmH5TDagJNBrZVBsY4tIXuv2VyqC11W4D8FeYMREr2%2FJE8
Ryrw94KmgoK4Y%2Bu3segiyULHXgdQGU0orSGYdmM6A9mf69SYt%2Bl7VT
j%2FA5WHw9ebZujx9ludqEHv86%2Bi1Zm4HlrwX5JD%2FqMRkilpp%2FJ9
Y3XfpbV8g1kM9lA75jLUejdnC8y6YjyDweVKO8si7vyOIUkgnzDtxJi3Bj
qdAU4pX%2FF9eF%2BiOLlpt1b5Oz6yVXPwKipJ41LEAQAAzshCZa%2FzhH
wGc%2BihnQFBSWjTvAsAgIBxEs%2BDVS%2BoEwHOcB0j%2ByXm2MfrTl8h
UKjaEF34eBSBPlGTk%2BiuifpAP4%2FXdItN9QCDTrcFAyOD%2BQXul%2F
d%2BzOzNkyvifrNDBtC%2FMJpBg1MQc4PZ0CbhK8hgZ8lAI2w9zlEK8ItF
WdF3e%2Fc%3D&X-Amz-Signature=b45a352fed1398fee0d4161953e3b
02d2c9af4f507a514bce9f886ad03ed78e7Copy to clipboardErrorC
opied
```


Even though the file is private, the pre-signed URL will allow temporary access for the time limit you set.

End Result

By the end of this exercise, you will:

- Have a versioned S3 bucket with multiple versions of an object.
- Be able to generate temporary, secure links for accessing objects in your private bucket.

4. Networking

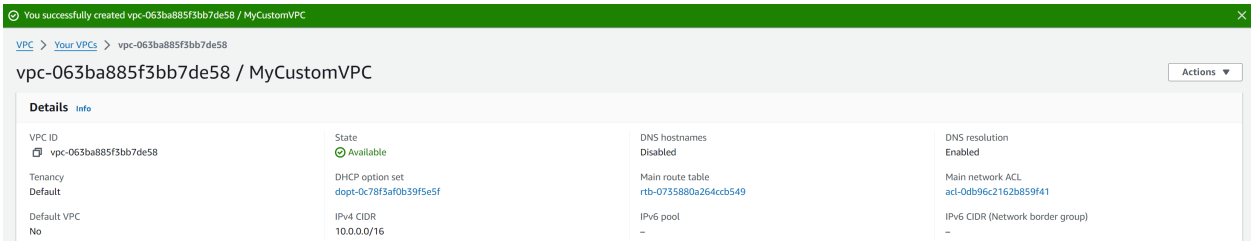
Lab: Creating a Custom VPC with Public and Private Subnets

Set up a custom VPC with both public and private subnets, an Internet Gateway for external traffic in the public subnet, and a NAT Gateway for internet access from the private subnet.

---

1. Create a VPC

1. Go to the **VPC Dashboard** in the AWS Console.
2. Click on **Create VPC**.
3. In the **VPC settings**, choose **VPC only** and enter:
    - **Name tag**: `MyCustomVPC`.
    - **IPv4 CIDR block**: `10.0.0.0/16`.
    - Leave everything else as default and click **Create VPC**.



---

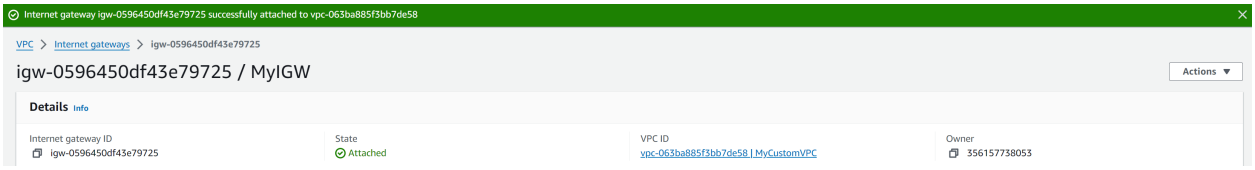
2. Create Subnets

1. Go to **Subnets** under the **VPC Dashboard**.
2. Click **Create subnet**.
3. In the **Create subnet** section:
    - Select the **VPC** you just created (`MyCustomVPC`).
    - **Subnet 1 (Public Subnet)**:
        - **Name**: `PublicSubnet`.
        - **Availability Zone**: Select `us-east-1a`.

    - **IPv4 CIDR block**: `10.0.1.0/24`.
    - Click **Add another subnet**.
  - **Subnet 2 (Private Subnet)**:
    - **Name**: `PrivateSubnet`.
    - **Availability Zone**: Use the same as the Public Subnet.
    - **IPv4 CIDR block**: `10.0.2.0/24`.
4. Click **Create subnet**.

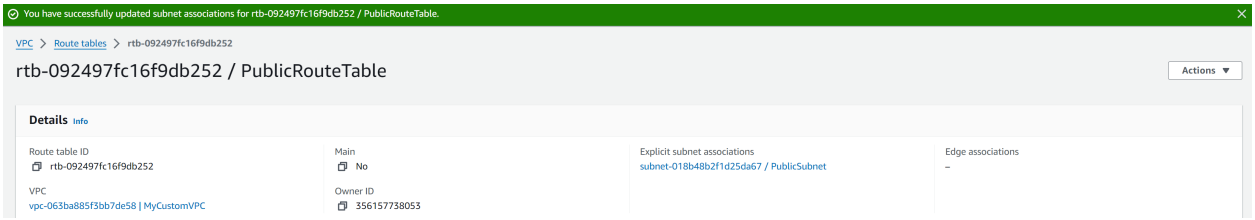
3. Create and Attach an Internet Gateway

1. Go to **Internet Gateways** under the **VPC Dashboard**.
2. Click **Create Internet Gateway**.
3. Provide the following details:
  - **Name tag**: `MyIGW`.
4. Click **Create internet gateway**.
5. After creation, select the new Internet Gateway and click **Actions > Attach to VPC**.
6. Choose the **VPC** (`MyCustomVPC`) and click **Attach Internet Gateway**.



4. Create a Route Table for the Public Subnet

1. Go to **Route Tables** under the **VPC Dashboard**.
2. Click **Create Route Table**.
3. Provide the following details:
   - **Name tag**: `PublicRouteTable`.
   - **VPC**: Select `MyCustomVPC`.
4. Click **Create route table**.
5. After creation, select the **PublicRouteTable** and go to the **Routes** tab.
6. Click **Edit routes** and add a route:
   - **Destination**: `0.0.0.0/0`.
   - **Target**: Select **Internet Gateway** and choose `MyIGW`.
7. Click **Save routes**.
8. Now go to the **Subnet Associations** tab.
9. Click **Edit subnet associations**, select **PublicSubnet**, and click **Save associations**.



---

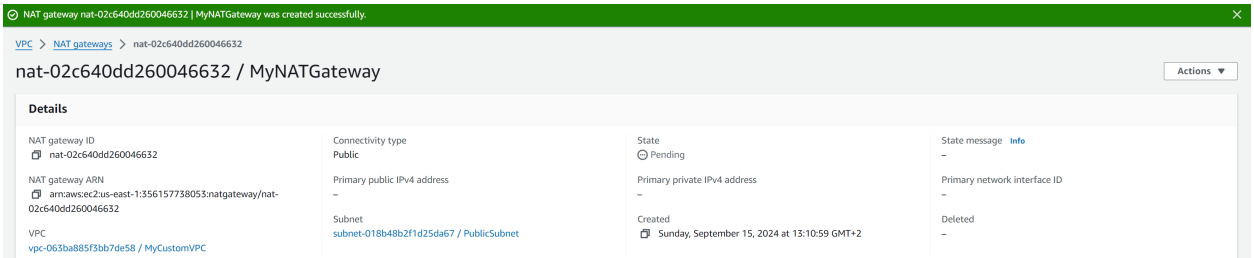
5. Create a NAT Gateway for the Private Subnet

1. Go to **Elastic IPs** under the **VPC Dashboard**.
2. Click **Allocate Elastic IP address** and then click **Allocate**.
3. Go to **NAT Gateways** under the **VPC Dashboard**.
4. Click **Create NAT Gateway** and provide the following details:
   - **Name tag**: `MyNATGateway`.
   - **Subnet**: Select `PublicSubnet`.

- **Elastic IP Allocation ID**: Select the Elastic IP you just allocated.

5. Click **Create NAT Gateway** and wait until it becomes available.

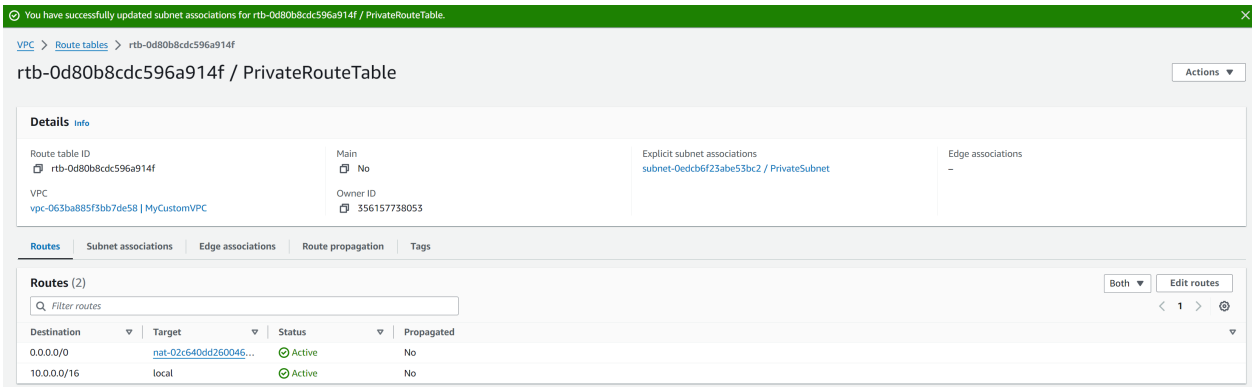
The NAT Gateway is placed in the Public Subnet because it needs direct internet access via the Internet Gateway to forward outbound traffic from instances in the Private Subnet. The Private Subnet cannot have direct internet access, so it routes traffic through the NAT Gateway in the public subnet, ensuring secure, outbound-only access to the internet.



6. Create a Route Table for the Private Subnet

1. Go to **Route Tables** under the **VPC Dashboard**.
2. Click **Create Route Table**.
3. Provide the following details:
   - **Name tag**: `PrivateRouteTable`.
   - **VPC**: Select `MyCustomVPC`.
4. Click **Create route table**.
5. After creation, select the **PrivateRouteTable** and go to the **Routes** tab.
6. Click **Edit routes** and add a route:
   - **Destination**: `0.0.0.0/0`.
   - **Target**: Select **NAT Gateway** and choose `MyNATGateway`.
7. Click **Save routes**.
8. Now go to the **Subnet Associations** tab.

9. Click **Edit subnet associations**, select **PrivateSubnet**, and click **Save associations**.



---

7. Test Connectivity

Step 1: Launch an EC2 Instance in the Public Subnet

1. Go to **EC2 Dashboard** and click **Launch Instance**.
2. Provide the following details:
   - **Name**: `PublicInstance`.
   - **AMI**: Select **Amazon Linux**.
   - **Instance Type**: `t2.micro` (Free tier eligible).
   - **Network settings**:
     - **VPC**: Select `MyCustomVPC`.
     - **Subnet**: Select `PublicSubnet`.
     - **Auto-assign Public IP**: Enabled.
   - **Security Group**: Create a new security group allowing **SSH (port 22)** from your IP address.
3. Launch the instance and wait for it to be running.
4. SSH into the instance using the public IP. Run:If you receive responses, it confirms that the instance in the **Public Subnet** has internet access.

```
ping google.comCopy to clipboardErrorCopied
```

```
$ ssh ec2-user@107.22.90.79 -i ~/Downloads/MyAWSKey.pem
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Sun Sep 15 11:30:55 2024 from 94.224.119.217
[ec2-user@ip-10-0-1-122 ~]$ ping google.com
PING google.com (142.251.179.100) 56(84) bytes of data.
64 bytes from pd-in-f100.1e100.net (142.251.179.100): icmp_seq=1 ttl=58 time=2.09 ms
64 bytes from pd-in-f100.1e100.net (142.251.179.100): icmp_seq=2 ttl=58 time=2.34 ms
^C
--- google.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 2.087/2.214/2.342/0.127 ms
[ec2-user@ip-10-0-1-122 ~]$
```

Step 2: Launch an EC2 Instance in the Private Subnet

1. Go back to the **EC2 Dashboard** and click **Launch Instance**.
2. Provide the following details:
   - **Name**: `PrivateInstance`.
   - **AMI**: Select **Amazon Linux**.
   - **Instance Type**: `t2.micro` (Free tier eligible).
   - **Network settings**:
     - **VPC**: Select `MyCustomVPC`.
     - **Subnet**: Select `PrivateSubnet`.
     - **Auto-assign Public IP**: Disabled.
   - **Security Group**: Create a new security group allowing **SSH (port 22)** from **ANY** address.
3. Launch the instance and wait for it to be running.
4. SSH into the **PublicInstance** (using its public IP) and then SSH into the **PrivateInstance** from the **PublicInstance** using the private IP of the private instance:

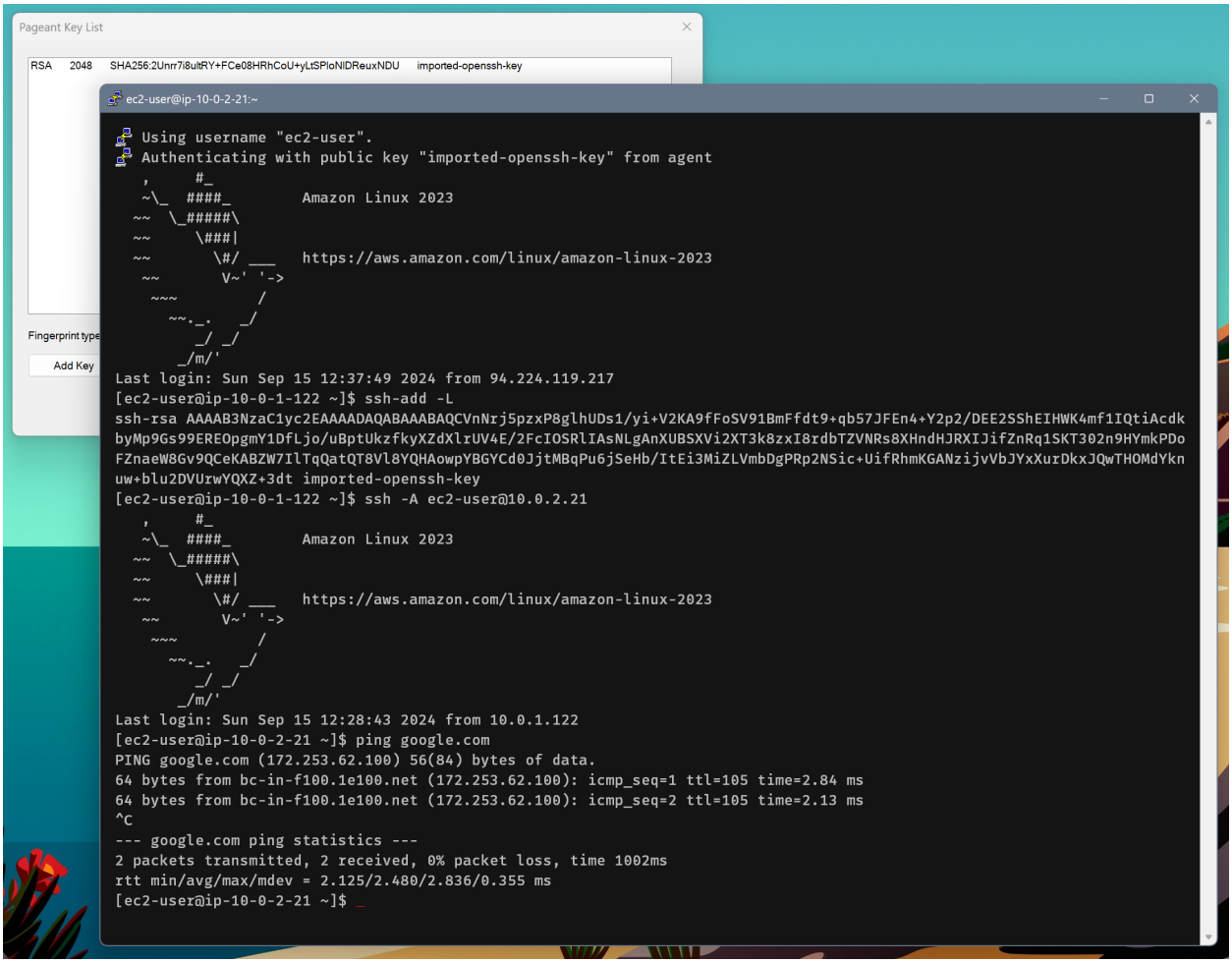
```
ssh ec2-user@<private-instance-private-ip>Copy to clipboar
dErrorCopied
```

5. Once inside the private instance, test internet connectivity by running:If you receive responses, it confirms that the instance in the **Private Subnet** has internet access via the **NAT Gateway**.

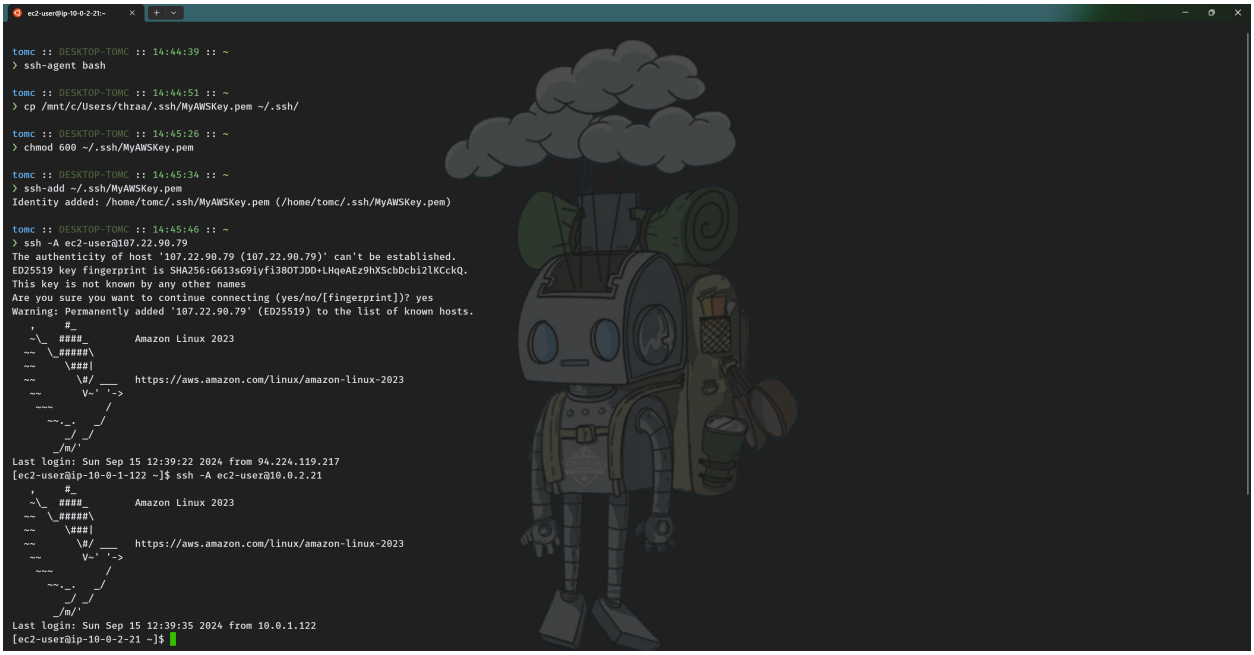
```
ping google.comCopy to clipboardErrorCopied
```

Familiarize yourself with ssh keys, `ssh -i`, `ssh-agent`, `ssh-add` using linux or linux on WSL2 and/or PuTTY, PuTTYgen, Pageant using Windows. You will need this knowledge for the rest of your studies and career.

Using Putty and Pageant:



Using Ubuntu on WSL2 `ssh-agent` and `ssh-add`:



Conclusion

- You've successfully created a **custom VPC** with separate **public** and **private subnets**.
- The public subnet has internet access via an **Internet Gateway**.
- The private subnet can securely access the internet via a **NAT Gateway**.

Exercises Networking I

Exercise 1: Configuring Security Groups for a Web Server

Configure security groups to control traffic to an EC2 instance acting as a web server.

Instructions:

1. **Launch an EC2 instance** in the `PublicSubnet`:
    - **Name**: `WebServerInstance`.
    - **AMI**: Amazon Linux 2.

- **Instance Type**: `t2.micro`.
- **Subnet**: `PublicSubnet`.
- **Auto-assign Public IP**: Enabled.

2. **Configure the security group** for the instance:
   - Allow **HTTP (port 80)** traffic from anywhere (`0.0.0.0/0`).
   - Allow **SSH (port 22)** only from your IP address.
3. **Install a web server**:
   - SSH into the instance and run:

     ```
     sudo yum update -y
     sudo yum install httpd -y
     sudo systemctl start httpd
     sudo systemctl enable httpdCopy to clipboardErrorCopied
     ```

   - Verify the web server is running by accessing the instance's public IP in a browser: `http://<public-ip>`.
4. **Test**: Ensure that the HTTP traffic is accessible publicly but SSH is restricted to your IP.

Exercise 2: Private Web Application in the Private Subnet

Create a web server in the private subnet and access it through a public EC2 instance.

Instructions:

1. **Launch an EC2 instance** in the `PrivateSubnet`:
   - **Name**: `PrivateWebInstance`.
   - **AMI**: Amazon Linux 2.
   - **Instance Type**: `t2.micro`.
   - **Subnet**: `PrivateSubnet`.

- **Auto-assign Public IP**: Disabled.

2. **Create a Security Group** for the private instance:
   - Allow **HTTP (port 80)** only from the security group of the `PublicInstance`.
   - Allow **outbound** traffic.
3. **Install a web server** on the private instance:
   - SSH into the `PublicInstance`, and then SSH into the private instance from there using its private IP.
   - Install the web server on the private instance:

     ```
     sudo yum update -y
     sudo yum install httpd -y
     sudo systemctl start httpd
     sudo systemctl enable httpdCopy to clipboardErrorCopied
     ```

4. **Access the private web server**:
   - From the `PublicInstance`, use `curl` to access the private instance's web server:

     ```
     curl http://<private-instance-private-ip>Copy to clipbo
     ardErrorCopied
     ```

This should display the default Apache or Nginx page from the private instance.

Exercises Networking II

Exercise 1 - Load & Scale your application

Configure your environment so you can use the AWS CLI. When the CLI is configured run the following command:

```
$ wget https://2tin-cloudessentials-resources.github.io/cloud
-essentials-labs/debuglabs/4_1.yml -outFile 4_1.yml
```

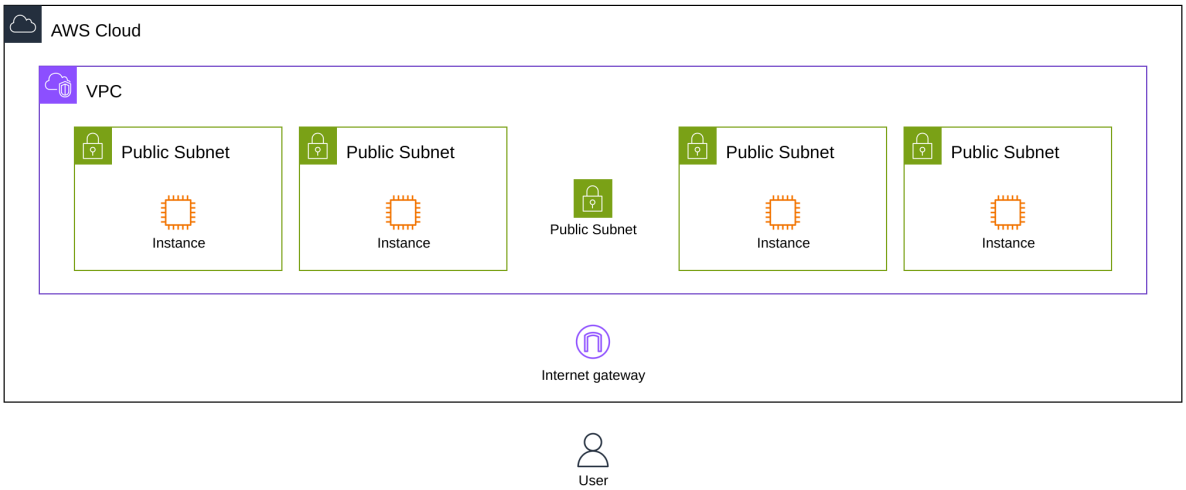
```
$ aws cloudformation deploy --template-file ./4_1.yml --stack
-name ch4ex1Copy to clipboardErrorCopied
```

This will set up an incomplete environment which you will be tasked to make highly available.

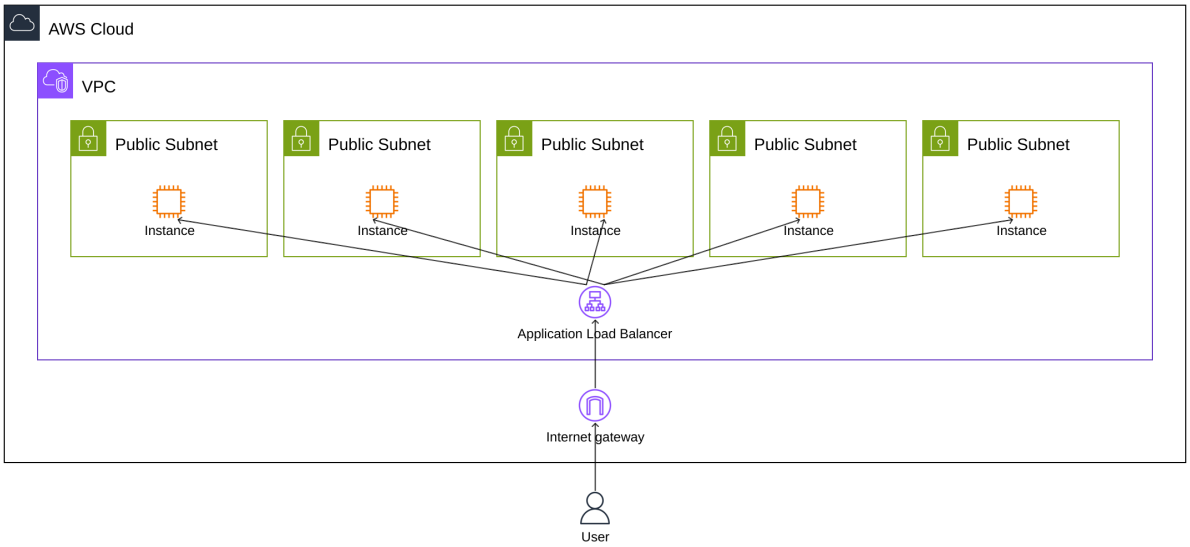
Scenario

This is what you should see, every EC2 should be accessible in the browser over http(80) and show a simple application with a distinct backgroundcolor.

You start with the following infrastructure:



The final state of the infrastructure is:



Task 1: Add the instances to the Load Balancer

Before we can create the load balancer we will need to create a **target group** which will be used to let the loadbalancer know where to assign incoming traffic to.

- Choose **Create target group**
- Choose a target type: **Instances**
- Target group name, enter: **MyLabGroup**
- Select **my-VPC** from the VPC drop-down menu.
- Choose **Next**. The Register targets screen appears.

  Note: Targets are the individual instances that will respond to requests from the Load Balancer.
- Add the 5 EC2s

Now we can go and create the actual Load balancer.

Task 2: Create a Load Balancer

- In the left navigation pane, choose **Load Balancers**.
- At the top of the screen, choose **Create load balancer**.

Several different types of load balancer are displayed. You will be using an **Application Load Balancer** that operates at the request level (layer 7), routing traffic to targets — EC2 instances, containers, IP addresses and Lambda functions — based on the content of the request.

- Under Application Load Balancer, choose **Create**
- Under Load balancer name, enter: **LabELB**
- For VPC, choose **My-VPC**

  You will now specify which subnets the Load Balancer should use. The load balancer will be internet facing, so you will select all 5 Public Subnets.

- Make sure a dedicate **Security group** is assigned that only allows web (port 80) traffic from the internet to the loadbalancer and another new Security group for the instances that **only** allows traffic from the loadbalancer to the web application.
- For the Listener HTTP:80 row, set the Default action to forward to **MyLabGroup**.
- Scroll to the bottom and choose **Create load balancer**

Task 3: Verify that loadbalancing is working

Go to the loadbalancer address in the browser and make sure you see the application switching background color when doing a hard-refresh (Ctrl-F5).

Finishing up

When you are done you can run the command below to destroy resources used by this exercise:

```
aws cloudformation delete-stack --stack-name ch4ex1
```